

Comparing Algorithms Algorithming the Maximum Algorithms Comparing Problem

by Author
student number: 123
E-Mail: Mail

Bachelor Thesis
Bachelor of Science
Major in Computer Science

First Supervisor:
Second Supervisor:
University

Date of Submission: Submission date

Abstract

The approximation of the Travelling Salesperson Problem is a widely studied field where even simple approaches seem to promise good results. In this thesis, we examine two different approaches to approximating the Maximum TSP. The first, a Graph Patch Heuristic, attempts to find a solution by patching a maximum-weight cycle cover into a singular cycle. The second, called the Tunnelling Algorithm, builds on the idea of approximating the Euclidean norm with a metric defined by a polytope. Despite their drastically different methodologies, we find that both approaches reduce to a similar sub-problem and yield similar approximation ratios. While the Graph Patch Heuristic seems to outperform the Tunnelling Algorithm with larger input sizes and dimensions, we find that the Tunnelling Algorithm actually solves the Max TSP in polynomial time for common norms such as L_1 and L_∞ .

This thesis expands on some of the nuances of both algorithms, presenting them in a self-contained manner.

Contents

1	Introduction	4
2	Greedy Patch Heuristic	4

List of Notations

n	Input size of a problem
d	Dimensions of a space
$\mathbb{R}^d$	d dimensional space of rational numbers
dim	Doubling dimension
$dist(\cdot, \cdot)$	Distance function
$G=(V,E)$	A graph being defined by it's set of vertices and edges
$\rho(n)$	Approximation Ratio
$\rho(n)_{GPH}, \rho(n)_{TA}$	Approximation Ratio of GPH and Tunnelling Algorithm
$\rho(n)'$	$\rho(n)' = 1/\rho(n)$; Approximation ratio that is ≤ 1
δ	Relative Error
$Approx$	Value-result of approximation algorithm
OPT	Value of optimal solution
PTAS	Polynomial-time approximation scheme
APX	Class of approximation algorithms defined in introduction
$\mathcal{O}(\cdot)$	Standard Big O-Notation
$\mathcal{P}$	Set of cities
GPH	Graph Patching Heuristic
k_c	Number of cycles in a cycle cover
$Loss(e(a_1, b_1), e(a_2, b_2))$	Weight loss through patching two edges together
C_0	Initial maximum-weight cycle cover
C	Current cycle cover
Tunnelling Algorithm	Name for Algorithm approximating max TSP
Tunnelling Max TSP	A specified version of the Travelling Salesman Problem
$\mathcal{H}$	Set of halfspaces vectors defining a polytope
h	A halfspace vector
$\mathcal{T}, T, T'$	Set of tunnels
t	A tunnel
E'	A singular tour
E''	A partial tour $E'' \subset E'$; informally a guessed outline
$F(c, t)$	Access distance from city to front of tunnel defined as $t \cdot c$
$B(c, t)$	Access distance from city to back of tunnel defined as $-t \cdot c$
$x_{i,j}^B, x_{i,j}^F$	Decision variable for back/front tunnel for city c_j and tunnel t_i
f_i^B, f_i^F	Number used front and back tunnels t_i within a guessed outline
d_i	Number of visits of tunnel t_i
$f = 2 \cdot k$	Number of facets on a polytope
k	Number of vectors defining a polytope; number of tunnels
L_n, L_1, L_2, L_∞	A few L-norms
$\mathcal{B}$	Ball or unit ball
$\mathcal{B}_x : x \in \{1, 2, l, g\}$	Different (unit) balls
λ	Shrink/Grow factor of a unit ball
$a \rightarrow b(c \rightarrow d)$	Equivalent to limit $\lim_{c \rightarrow d} a = b$

1 Introduction

Since the relationship between the set of Problems in P , solvable in polynomial time, and Problems in NP , the set of problems where the solutions can be verified in polynomial time, is unclear, there exists great interest in finding approximation algorithms solving NP and NP-hard problems.

An approximation algorithm is supposed to guarantee a better runtime, with the trade-off that the solution may not be optimal. Therefore, there are at least two factors to compare approximation algorithms by, runtime and how close the solution is to the optimal solution. There are multiple ways to evaluate the accuracy of an algorithm, in some cases it can be an additive factor, but most commonly, the accuracy is measured with an approximation ratio $\rho(n)$, where the value of a solution *Approx* provided by an approximation algorithm is within a factor of $\rho(n)$ to the actual optimal value *OPT* for any input size n . Conventionally $\rho(n) \geq 1$ by defining it as $\frac{OPT}{Approx}$ for maximization problems and $\frac{Approx}{OPT}$ for minimization problems, or generalizing to $\max(\frac{OPT}{Approx}, \frac{Approx}{OPT}) \leq \rho(n)$ algorithms. Some authors, such as Shenmaier¹ have used a relative error δ , where $\delta = \frac{OPT - Approx}{OPT}$ to describe the accuracy of the approximation algorithm. The APX class is a category of algorithms that approximate an optimisation problem in NP in polynomial runtime with an approximation ratio that is constant.

2 Greedy Patch Heuristic

We will start with the simpler one of the two algorithms that finds a tour by first finding a maximum-weight cycle cover and then combining it into one cycle creating a tour. The results of this section are mainly focused on the paper by Shenmaier(2022)¹, where he describes an algorithm for which he estimates a constant approximation ratio, but the more interesting result is that he proves a relative error that improves with input size. The algorithm itself is a modified version of the greedy Karp-Steel algorithm where Shenmaier¹ references papers that analyse such greedy Karp-Steel algorithm in the context of Minimum TSP. For more exact steps of the algorithm we will mainly use details from a 1999 paper by Clover et al.², where we will of course modify the proposed algorithm to work for the Maximum TSP. Additionally the Hungarian Algorithm from

...

will find its use in this algorithm. Tracking down the references of various papers discussing such a patching algorithm we can find that the idea dates back to at least a 1976 paper by Karp³, as well as another paper we consider from 1985 by Karp and Steel⁴.